

Project Report: Week 4 Summary & Deep Learning Revision

1. Convolutional Neural Networks (CNNs)

Designed primarily for spatial/image processing by extracting layered hierarchies of local visual features (edges → complex → relations).

- **Kernels & Operations:** Grids of weight matrices overlaid across an input layer. The calculated output over a center pixel maps to:

$$\sigma_p = \sum_{p=1}^P W_p \cdot \text{pixel}_p$$

- **Padding:** Adds extra boundary pixels around data frames to eliminate the "edge effect" and preserve spatial dimensions.
 - **Stride:** The step size moving the kernel across the input. A stride > 1 downscales the output grid.
 - **Depth & Pooling:** Depth dictates input channels (e.g., RGB = 3). Pooling operations (like Max-Pooling) systematically shrink physical tensor dimensions without adding learnable parameters.
-

2. Recurrent Neural Networks (RNNs)

Specifically engineered to process continuous sequential and temporal time-series records by introducing an internal loop to preserve historical context.

- **Hidden State & Unfolding:** RNNs hold a hidden state which passes information across time. Unfolding expansions across sequence steps enable **Backpropagation Through Time (BPTT)** to calibrate weight matrices.
- **Shared Weight Matrices:** Unlike standard feedforward networks, RNNs use shared weights (U, W, V) across all temporal intervals to process variable sequence lengths.

Hidden State: $h_t = \sigma(U \cdot X_t + W \cdot h_{t-1} + B)$

Output: $Y_t = \text{Output_Activation}(V \cdot h_t + C)$

3. Long Short-Term Memory (LSTMs)

LSTMs resolve the classic RNN **vanishing/exploding gradient problem** by introducing an explicit **Memory Cell State** regulated by three unique mathematical gates:

- **Forget Gate** : Regulates how much historical cell memory to drop.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **Input Gate** : Dictates what new information to write into the cell.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **Output Gate** : Selects which hidden internal segments pass to the next recurrent node.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

4. Deep Learning Code Revision (TensorFlow / Keras)

This code revision focuses on assembling a stacked multi-layer LSTM model using the Keras Sequential API to ingest sequential tabular inputs.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 1. Constructing Mock Sequential Inputs
X = np.random.random((100, 10, 1)) # (Samples, Time Steps, Features)
y = np.random.random((100, 1))      # Target continuous variable

# 2. Structural Layer Assembly
model = Sequential([
```

```

    LSTM(50, return_sequences=True, input_shape=(10, 1)),
    LSTM(50, return_sequences=False),
    Dense(1)
])

# 3. Setting Up Optimization Routines
model.compile(optimizer='adam', loss='mean_squared_error')

# 4. Neural Network Execution
model.fit(X, y, epochs=10, batch_size=16)
model.summary()

```

Code Term Breakdown:

- `np.random.random((100, 10, 1))` : Prepares a 3D NumPy array tensor. Deep learning sequence models strictly demand inputs formatted exactly as: `(batch_size, time_steps, features_per_step)` . Here, it simulates 100 independent sequence records tracking a window of 10 sequential timesteps across 1 individual indicator feature.
- `Sequential([...])` : Initializes a linear stack blueprint container to sequentially layer structural neural layers.
- `LSTM(50, return_sequences=True)` : Allocates an LSTM layer with 50 internal hidden memory blocks. `return_sequences=True` forces the layer to output its hidden state for *every* individual input time step, creating a 3D tensor output. This is a mandatory parameter whenever stacking another LSTM layer directly afterward.
- `LSTM(50, return_sequences=False)` : The final recurrent layer. `return_sequences=False` instructs the layer to return only the final hidden state vector at step $t=10$, flattening the output into a 2D tensor ready for standard dense mapping layers.
- `Dense(1)` : A fully connected layer that maps the 50-dimensional output down to a single continuous scalar output node.
- `model.compile(optimizer='adam', ...)` : Configures learning setups. `'adam'` leverages an adaptive moment estimation algorithm to dynamically scale individual parameter learning rates, and `'mean_squared_error'` evaluates model prediction variance against targets.
- `model.fit(..., epochs=10, batch_size=16)` : Commences model training. The network steps through the full data allocation 10 separate iterations

(`epochs`), adjusting network weights through backpropagation after processing mini-batches of 16 rows at a time.
